

第 7 章 面向对象设计方法 · 期末复习讲义（开卷 A4 打印版）

本章速览

一句话主线：以第 4 章用例模型、领域模型为输入，运用 UML 把"系统对外做什么"转换成"系统内部由哪些软件对象协作来做"——**先得动态结构（用例实现的顺序图/协作图），再归纳静态结构（设计类图）。**

核心三件事：①软件体系结构设计（分层）②用例实现方案设计（交互图 + GRASP）③用户界面设计。

四个高频考点：①五层架构 + 持久化层 DBFacade ②GRASP 三大模式（信息专家 / 创建者 / 控制器）③据系统事件画设计交互图、推导方法 ④据对象行为画状态迁移图。

题目关键词导航（看到这些词就回到本章）

关键词 / 信号	考点定位	本章定位
软件设计模型 / 交互图 / 顺序图 / 协作图	用例实现（动态结构）	必考大题
GRASP / 控制器模式 / 信息专家 / 创建者	职责分配	必考大题/简答
外观控制器 / 用例控制器 / 臃肿	控制器分类判别	判断/单选
DBFacade / IMapper / Mapper / 持久化层	持久化层设计	大题/简答
GUI/控制器层/应用逻辑层/持久化层/DB	B/S 分层结构	大题/简答
设计类图 / 类的方法 / 推导属性	静态结构	综合题
状态迁移图 / 子状态 / 调度	对象状态图	综合题
SSD vs 设计顺序图 / 操作契约后置条件	分析与设计边界	简答/判断
OOA 与 OOD 是否采用一致的表示法	OOA/OOD 关系	判断/简答
开闭原则 / 依赖倒置 / 单一职责	设计原则（背景知识）	判断/单选

一、知识主线（必背骨架）

OOD 总过程

用例模型 + 领域模型 + 体系结构框架

↓（GRASP + 分层）

动态结构：每个用例的交互图

↓（扫描所有交互图）

静态结构：设计类图（类/属性/方法/关联）

↓

必要时补：状态图/活动图/详细设计

口诀：先体系结构、再动态、后静态；先交互、后类图。

输入 → 输出对照

输入（OOA）	输出（OOD）
用例图/用例描述	分层设计类图（静态）
系统顺序图 SSD	设计顺序图/协作图（动态）
操作契约	对象的方法/属性
领域模型（概念类）	业务/领域层软件类
—	新增：控制器类、DBFacade、Mapper、UI 类
—	状态图（针对复杂对象）

章节逻辑闭环

1. 分层（7.2）：把软件切成 5 层，限制消息方向。
2. 分配职责（7.3.1~7.3.4）：GRASP 三大模式回答"谁接收、谁创建、谁拥有方法"。
3. 用例实现（7.3.5）：每个系统事件画一张交互图。
4. 持久化层（7.3.6）：DBFacade + IMapper + Mapper + DBConnection。
5. 设计类图（7.3.7）：扫描所有交互图，归纳静态结构。
6. startUp（7.3.8）：初始化领域对象。

二、核心知识点详解（PPT 主导）

2.1 面向对象设计综述

定义：以需求分析阶段得到的**用例模型 + 领域模型**为基础，运用 UML 构建软件系统结构，通过一系列设计模型说明用例的实现过程。

三方面工作（必背）：

1. 软件体系结构设计——选择合适的软件架构（如 B/S 分层、MVC）。
2. 用例实现方案设计——通过 UML 交互图描述每个用例的实现过程。
3. 用户界面设计——可前移到需求阶段，本章不是重点。

主要设计活动（PPT 原文）：**选择体系结构 → 用交互图描述用例实现 → 给出以类图表示的系统静态结构 → 按设计原则优化。**

易考：简答"OOD 需要做哪些工作及其成果模型"。答：①体系结构设计（包图）②用例实现（交互图 + 类图）③复杂对象状态图、流程化活动图。详见 § 开放题模板第 1 题。

软件概要设计步骤：

1. 选择合适的软件架构。（单机？CS 或 BS？云服务？）
2. 系统的动态结构设计：
 - 针对用例 SSD 中的每个系统事件，使用 Sequence diagram/ Collaboration diagram 给出符合该系统事件操作契约的内容。
 - 如果软件对象具有多种不同职责，需要使用 State machines diagram 进行状态迁移设计。
3. 系统的静态结构设计：
 - 对所有用例或子系统级别的用户交互图归纳，使用 Class diagram 给出系统的静态结构。
4. 软件详细设计：
 - 针对系统静态结构中的每个方法，使用 Activity diagram 进行逻辑结构的设计。
5. 软件架构：
 - 三种基础架构：单机、CS 和 BS、云服务。
 - 服务器端的分层：控制器层+应用逻辑层（必须）+基础架构层+持久化层+DB（必须）

面向对象设计的关键步骤：在确定软件框架结构的基础上，进行如下内容设计：

1. 发现对象（发现软件类）：根据需求和选择的架构和模式确定系统由哪些对象构成。
2. 确定对象属性：明确对象特征属性。
3. 确定对象行为：明确对象功能职责。
4. 确定对象之间的关系：根据 SSD 和操作契约、选择的架构和模式，明确系统如何相互协作完成功能需求的交互过程。

2.2 模型层次化（5 层架构）

分层是面向对象基于软件体系结构风格的一种方案选择，符合面向对象设计原则，使系统易于扩展和维护。

层名	职责	典型类
用户界面层	定义所有角色功能操作的界面表现形式；接收用户输入、显示返回结果	«UI» XxxWindow
控制器层	协调、控制其他类共同完成用例功能；接收系统事件并委派	XxxController
业务/应用层	实现用例要求的系统级核心功能（信息专家在此）	Sale、Order、Patient、Prescription
持久化层	保存需持久化的数据对象，封装存储技术	DBFacade、XxxMapper、IMapper
系统层	封装操作系统相关功能；增加可移植性	«system» IPC、文件流类

消息方向约束（教材 7.2 原文）：

- 同层可协作。
- 相邻层可单向协作。
- UI → 控制器 → 业务逻辑 → 持久化 → DB。
- 禁止：业务对象调用 UI；GUI 直接访问 DB。
- 控制器可跨业务逻辑层直达持久化层（适度灵活性）。

常考：分层好处=健壮性+可扩展+可维护+可移植。回答 5 层结构要按顺序写全。

2.3 各层对象设计原则（核心简答）

① 控制器层（7.2.1）

问题：SSD 中的系统事件由哪个对象接收？

方案一：直接发给业务对象（不推荐，UI 与业务耦合）。
方案二：由一个控制器对象接收并转发——推荐，称为**控制器类**。每个用例通常设计一个控制对象。

② 业务/应用层（7.2.2）

设计依据：参考**领域模型**中的**概念类**进行定义和命名。OOA 关注概念本质与属性；OOD 在此基础上增加操作、调整关系，使之成为**软件类**。
结论：OOA 与 OOD 采用一致的表示法，两者无缝衔接，不存在结构化方法中的“分析与设计的鸿沟”。

③ 持久化层（7.2.3 + 7.3.6）

问题：业务数据如何保存到数据库？

方案：引入**持久化对象**，专门负责 OOC CRUD（创建/检索/更新/删除）。
为何不交给领域对象自己持久化：①领域对象与存储机构强耦合；②违反 SRP（技术服务与应用逻辑混合）。

④ 系统层（7.2）

封装 OS 系统调用（文件流、IPC、审计日志），上层业务逻辑直接访问系统类而不直接访问系统调用，便于跨平台移植。

2.4 GRASP 总览（必考）

GRASP = General Responsibility Assignment Software Patterns，共 9 个模式。

模式	核心问题
信息专家	给对象分配职责的通用原则
创建者	谁负责创建某实例
控制器	谁第一个接收系统事件
低耦合	降低模块间依赖
高内聚	职责功能聚焦
多态 / 纯虚构 / 中介者 / 受保护化	见教材，本章略

两类职责：

- **了解型 (knowing)**：①私有封装数据；②相关联对象；③可派生/计算的事物。
- **行为型 (doing)**：①自身行为（创建/计算）；②启动其他对象动作；③控制/协调其他对象活动。

核心句（教材原文）：**面向对象设计最关键的活动之一是正确地给对象分配职责**，这直接关系到设计模型的质量。

职责的内聚（自己只干自己的事）：目的是提高内聚降低耦合，减少不必要的关联关系

2.5 控制器模式（7.3.2）

问题：第一个接收系统事件的软件对象是什么？

解决方案：**把接收/处理系统事件的职责分配给控制器层对象**。分两类：

类型	命名	适用
外观控制器 Facade	系统/设备名（如 Register、ATM、System）	系统事件不多；UI 不能重定向事件
用例控制器 Use Case	<用例名>Controller（如 TakeTestController）	外观控制器变“臃肿”；用例复杂

关键铁律（必考判断）：**不论外观控制器还是用例控制器，都只接收系统事件消息，不实现系统操作的内容**。系统操作应委托给领域对象处理。

记忆：控制器 = 协调员，不是工程师。**一个用例一个控制器（用例控制器情况下），同一用例的所有系统事件用同一个控制器实例。**

2.6 创建者模式（7.3.3）

问题：哪个对象负责创建类 A 的实例？**操作契约中对象实例的创建**

B 创建 A 的 4 个判定条件（满足其一即可）：

1. B 聚合或包含 A；
2. B 记录 A；
3. B 密切使用 A；
4. B 拥有创建 A 所需的**初始化数据**（B 是 A 的信息专家）。

思想：**体现低耦合，对迪米特法则（LKP）的具体运用**。

示例：Sale 创建 SalesLineItem（Sale 包含 SalesLineItem）；Prescription 创建 PrescriptionItem；Order 创建 OrderItem；Rule 创建 RuleItem。

注意：若创建逻辑复杂（性能缓存、根据属性从一族类中选），改用**工厂方法模式**，将创建职责委托给“工厂”辅助类。

2.7 信息专家模式（7.3.4）给对象分配职责的通用原则

问题：每个软件对象的方法如何确定？

解决方案（教材原话）：将职责分配给拥有履行职责所必需信息的类。即**对象处理自己拥有信息的事务**。

方法推导铁律：根据交互图上的对象，**接收消息的对象一定具有处理该消息的职责和能力**，根据消息的名称和参数定义同名方法。

查找步骤：①先到设计模型找相关类；②设计模型没有，再到领域模型找/扩展。

POS 机/考试系统典型分配：

- Sale 拥有 SalesLineItem 集合 → 负责 calcTotal()。
- SalesLineItem 拥有 quantity 与 ProductDescription → 负责 calcSubtotal()。
- TestPaper 拥有 Question 集合 → 负责 getScore()。
- Question 拥有答案与评分依据 → 负责抽象方法 getSubScore()。

边界：当某类按信息专家分配到大量不同类型职责时违反 SRP（如 Employee 自己持久化），应将技术服务职责隔离到专门类（如 EmployeeMapper）。

2.8 用例实现的设计过程（7.3.5）

定义：用例实现 = 在设计模型中描述分层结构中相互协作的软件对象如何实现用例的所有成功/失败场景。

设计步骤（必背 6 步）：

- 1. 针对系统事件，确定分层结构各层必需的**软件对象**；
- 2. 确定软件对象必须执行的**交互过程**（消息序列）；
- 3. 确定每个软件对象必须具有的方法；
- 4. 确定用例实现的具体细节；
- 5. 针对多方法对象确定**状态迁移图**；
- 6. 汇总所有用例，得到软件系统的动态结构。

顺序图 vs 协作图：

- 顺序图：直观表达时间顺序，对象多时画不下。
- 协作图：节省空间，必须带**消息编号**（1、1.1、1.2、1.2.1）。

综合题铁律：系统事件 → 控制器接收 → 委托信息专家处理 → DBFacade/DAO 持久化 → 返回结果。生命线至少包含 GUI / Controller / 业务对象 / DBFacade。

2.9 持久化层设计（7.3.6 重点）

5 个核心对象：

对象	职责
DBFacade	持久化层对外的统一服务窗口；维护领域类与 Mapper 类的对应关系；提供 insert/delete/update/get/gets
IMapper (IBean)	Mapper 公共接口，抽象出 CRUD 操作
XxxMapper / XxxBean	负责某类领域对象的 CRUD（如 StudentMapper、SaleBean）
DBConnection	负责数据库连接建立/释放（隔离易变部分）
DB	实际数据库

DBFacade 5 个操作签名（必背）：

```
boolean insert(Object obj, Class)
boolean delete(String condition, Class)
boolean update(Object obj, Class)
Object get(String condition, Class)
List gets(String condition, Class)
```

设计遵循 OCP：DBFacade 仅依赖 IMapper 接口，新增领域类只需新增 Mapper 子类，DBFacade 无需修改。

2.10 创建设计类图（7.3.7）

6 步骤：

- 1. 扫描所有交互图 + 领域模型涉及的类，**识别软件类**；
- 2. 把领域模型中已识别的属性添加到类中；
- 3. **根据交互图为软件类添加方法**（接收消息 = 方法；忽略 create 构造、简单 get/set）；
- 4. 添加更多类型信息（属性类型、方法参数、返回类型）；
- 5. 添加关联和导航（A→B 导向的情况：A 发消息给 B / A 创建 B / A 维护到 B 的连接）；
- 6. 类成员细节（可见性、方法体，可选）。

关键转化规则：

- **组合/聚合/关联** → 对象引用或集合属性。
- **多对象消息**：方法是**容器对象**的方法，不是每个子对象的方法。
- **create 消息**：忽略（构造函数）。
- **get/set**：忽略。

2.11 startUp 用例（7.3.8）

原则：startUp 虽最早执行，但**最后设计**——保证初始化活动被充分挖掘。

初始化对象选择：

- 领域对象的**容器**；
- 聚集层次结构的**根或临近根**的类；
- 可能是外观控制器；
- 包含所有/大多数其他对象的对象。

POS 机实例（Store.startup）：**选择 Store 为根对象 → 创建 Store/Register/ProductCatalog/ProductDescription → 建立关联。**

2.12 复杂对象状态图（7.3.5 第 5 项）

针对具有**多个方法、多种状态、调度策略**的软件对象绘制。

典型对象：温控模块、自助挂号机、ATM、考试控制器。

必含要素：

- 起始状态（实心圆）和结束状态（环内实心圆）；
- 主要稳定状态（一级状态）；
- 复杂对象用**子状态**（二级状态）；
- 迁移事件/条件写在箭头上。

大题套路：识别对象 → 找一级状态（如空闲/工作）→ 工作下分子状态（资源分配/调度/等待）→ 标注调度条件（请求数 vs 资源数 M、风速比较等）。

三、补充知识点（教材独有）

3.1 OOD 三大目的（教材 7.1）

- 1. 使反映问题域本质的总体框架和组织结构长期稳定，细节可变；
- 2. 把稳定部分（问题域）和可变部分（与实现有关）分开；
- 3. 分析结果重用，同一分析可用于不同设计/实现。

3.2 良好分层结构的判定

- 修改某一层不影响其他层（易扩展/维护）；
- 层是模块化的，可重写或替换整层（只要接口不变）→ 可移植；
- 同层可协作；相邻层单向协作；消息单向流动减少耦合。

3.3 OOCRUND 概念

面向对象实例创建、检索、更新和删除（OOCRUND）。持久化层对外提供的就是这 4 类操作。消息从业务逻辑层发往持久化层。

3.4 设计模式 4 要素

- 1. **模式名称**（助记名）；
- 2. **问题**（何时使用）；
- 3. **解决方案**（组成部分、相互关系、职责协作）；
- 4. **效果**（应用效果与权衡）。

GRASP 共 9 模式：信息专家、创建者、低耦合、高内聚、控制器、多态、纯虚构、中介者、受保护化。

3.5 设计模型与领域模型的关键差异

维度	领域模型	设计模型
类性质	概念类	软件类
是否含方法	否	是（参数、返回值）
关系	概念性关联	导航、多重性、依赖、实现
新增类	—	控制器、DBFacade、Mapper、UI
用途	理解问题域	直接指导编码

3.6 教材实例：在线考试系统

8 个系统事件（必考案例）：

```
logon(role, Id, pwd)
getAvailableTestPapers(stuId)
selectTestPaper(courseName)
logonTestPaper(pwd)
startTest()
submitAnswer(questionId, answer)
getNextQuestion()
endTest()
```

控制器选择：因 20 个用例、每个用例多事件，外观控制器会"臃肿"，故选用**用例控制器**——LogonController 处理登录用例、TakeTestController 处理考试用例。

设计类图分层（教材 7.3.7）：

- UI 层：mainLogon、mainWindow、showTestPaperWindow、TestPaperLogon、TestPaperWindow («UI»）。
- 控制器层：LogonController、takeTestController。
- 领域层：Student、TestPaper、Question（抽象）、Test、Rule、RuleItem、QuestionCatalog、QuestionSpecification。
- 持久化层：DBFacade、DBConnection、IMapper + 8 个 Mapper 子类。

3.7 POS 机实例（Register 外观控制器）

架构：B/S 分层，至少体现控制器层和应用逻辑层。

4 个系统事件：makeNewSale、enterItem、endSale、makePayment。

控制器选择：系统事件少，选**外观控制器 Register**（注意：此时 Register 已不是物理终端设备）。

关键职责分配：

- Register：接收所有系统事件、创建 Sale、协调；
- Sale：创建 SalesLineItem（创建者）、calcTotal()（信息专家）；
- SalesLineItem：calcSubtotal()（拥有 quantity + ProductDescription）；
- ProductCatalog：包含 ProductDescription → 负责查找。

四、核心对比表

表 1 外观控制器 vs 用例控制器（高频判断/简答）

维度	外观控制器 Facade	用例控制器 Use Case
代表	整个系统/设备	一个用例场景
命名	系统名/设备名 (Register、ATM、System)	<用例名>Controller (TakeTestController)
适用条件	系统事件不多；UI 不能重定向事件	外观控制器臃肿；用例复杂
实例数	通常系统级一个	每用例一个且仅一个
状态维护	不便维护用例状态	便于维护用例状态、有执行顺序的系统操作
共同点	都只接收/委派系统事件，不实现系统操作内容；都是非 UI 对象	

表 2 OOA 模型 vs OOD 模型（产出对比）

维度	OOA（分析阶段）	OOD（设计阶段）
关注点	问题域、系统职责	如何把职责映射到软件功能
输入/输出	需求 → 分析模型	分析模型 + 实现约束 → 设计模型
主要图	用例图、领域类图、SSD、操作契约、状态图	设计顺序图/协作图、设计类图、组件图、包图、详细活动图
类性质	概念类（无方法）	软件类（有属性、方法、可见性）
是否含实现细节	否	是（架构、语言、数据库）
关系	采用一致的面向对象概念和 UML 表示法，无缝衔接	

表 3 SSD vs 设计顺序图

项	SSD（系统顺序图）	设计顺序图
所属	用例模型（OOA）	设计模型（OOD）
生命线	Actor + 系统黑盒	GUI/Actor + Controller + 业务对象 + DBFacade + DB
消息	系统事件 (Actor↔System)	对象间方法调用
用途	定义系统操作和操作契约	用例实现，推导对象方法

表 4 顺序图 vs 协作图

项	顺序图 Sequence	协作图 Collaboration
组织维度	时间（自上而下）	对象间连接
消息编号	位置即顺序，可不编号	必须编号（1, 1.1, 1.2...）
优点	直观	节省绘图空间
何时选	对象较少、强调顺序	对象较多、图纸有限
等价性	UML 中两者语义等价，可互转	

表 5 GRASP 三大模式速查

模式	问题	方案	口诀
控制器	谁接收系统事件	位于控制器层的对象（外观/用例）	协调员不干活
创建者	谁创建 A 实例	B 聚合/记录/密切使用/拥有初始化数据	谁装谁造
信息专家	谁有方法 M	拥有履行职责所必需信息的类	接收消息者有该方法

五、开放题答题模板（必背）

题 1 "OOD 需要做哪些工作及其成果模型？"

答：主要包括三方面工作：

1. **软件体系结构设计**——参考成型框架，针对核心用例迭代验证。成果：**包图**。
2. **用例实现方案设计**——分动态与静态：
 - 动态：针对每个用例 SSD 中的系统事件画交互图，复杂对象画状态图。成果：**顺序图/协作图、状态图**。
 - 静态：扫描所有交互图归纳分层设计类图。成果：**设计类图**。
3. **用户界面设计**（可与上面并行）。
4. **详细设计**：每个对象方法用**活动图/程序流程图**描述内部逻辑。

题 2 "为何引入持久化层？"

答：

1. **封装变化**：数据库会升级、迁移、改模式，持久层将变化局部化。
2. **单一职责**：领域对象若自己持久化会违反 SRP（技术服务与应用逻辑混合）。
3. **解耦**：领域对象与存储机构强耦合，引入持久层后业务逻辑不直接依赖数据库。
4. **可维护/可扩展/可移植**：无论存储策略如何变化，业务领域类都不受影响。
5. **统一入口**：DBFacade 简化其他层访问，仅依赖 IMapper 接口满足 OCP。

题 3 "控制器层对象的设计原则？"

答：

1. 系统事件由位于控制器层的非 UI 对象接收，UI 仅产生事件并委托。
2. 分类：**外观控制器**（系统/设备名）和**用例控制器**（<用例名>Controller）。
3. 系统事件少或 UI 不能重定向 → 外观控制器；外观控制器臃肿或用例复杂 → 用例控制器。
4. 同一用例的所有系统事件用同一个控制器实例（便于维护用例状态）。
5. **控制器只接收和委派，不实现系统操作内容**，业务逻辑由领域对象（信息专家）承担。

题 4 "如何根据系统事件绘制设计交互图？"

6 步法（套用即可）：

1. **识别系统事件**：从 SSD/操作契约中找出，写明参数。
2. **选控制器**：简单系统用外观控制器，复杂用例用 <用例名>Controller。
3. **确定业务对象**：从领域模型转化概念类，补足必要对象。
4. **分配职责**：控制器接收、信息专家处理、创建者创建、DBFacade 持久化。
5. **画图**：生命线左→右为 Actor/GUI→Controller→业务对象→DBFacade→DB；消息自上而下；带关键参数；重要返回用虚线。
6. **说明 GRASP**：写明控制器模式、信息专家模式、创建者模式的依据。

题 5 "如何绘制对象状态迁移图？"

5 步法：

1. 明确对象（控制器/模块/设备），不画业务流程。
2. 找稳定状态（一级状态，如空闲、工作）。
3. 复杂行为下分子状态（如资源分配、调度、等待）。
4. 在箭头上标注触发事件/条件。
5. 覆盖异常、超时、资源不足、调度策略；画起始/结束状态。

题 6 "领域模型类图与设计模型类图有何不同？"

见 §四 表 2 + 表 5 同名行。要点：概念类 vs 软件类、无方法 vs 有方法、不含实现 vs 含实现细节、来源（名词短语）vs 来源（领域模型 + 新增设计类）。

题 7 "OOA 与 OOD 阶段有何不同？面向对象方法相对结构化的优势？"

不同：分析阶段针对问题域，把需求转化为分析模型；设计阶段考虑实现，产生可编码的设计模型。

优势：OOA 与 OOD 采用**一致**的面向对象概念与 UML 表示法，从 OOA 到 OOD **不存在转换**，紧密衔接；结构化方法中分析与设计之间存在鸿沟。

题 8 "OOAD 已知条件与设计结果？"

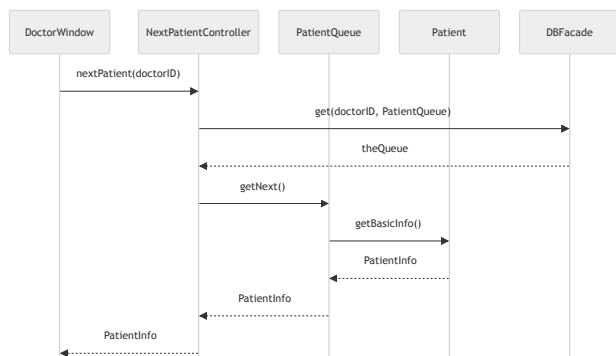
已知（输入）：用例模型（4 要素）、领域模型、软件体系结构框架、面向对象设计原则与设计模式。

结果（输出）：动态结构（交互图）+ 静态结构（设计类图/组件图/包图）；必要时含状态图、活动图。

六、典型 UML 图重绘（Mermaid）

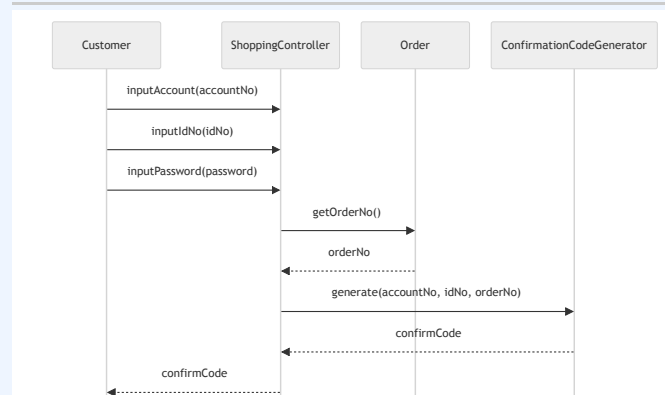
图 1 nextPatient(doctorID) 设计顺序图（高频大题）

考点：信息专家——PatientQueue 知道队列，Patient 知道自身信息。



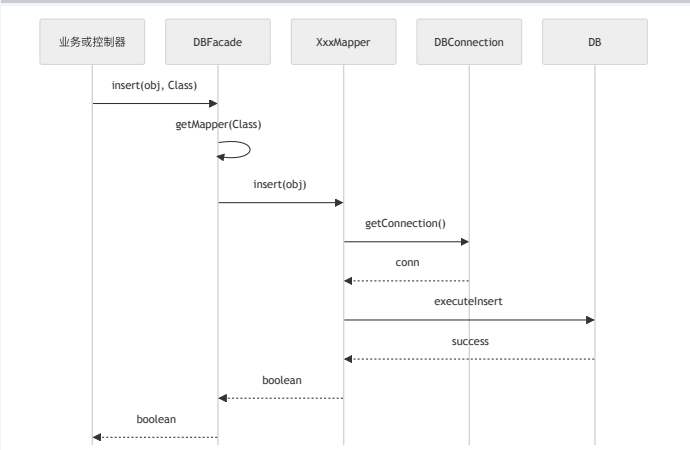
对应方法：NextPatientController.nextPatient(doctorID)、PatientQueue.getNext()、Patient.getBasicInfo()、DBFacade.get(cond,Class)。

图 2 网银付款生成确认码（信息专家 + 创建者）



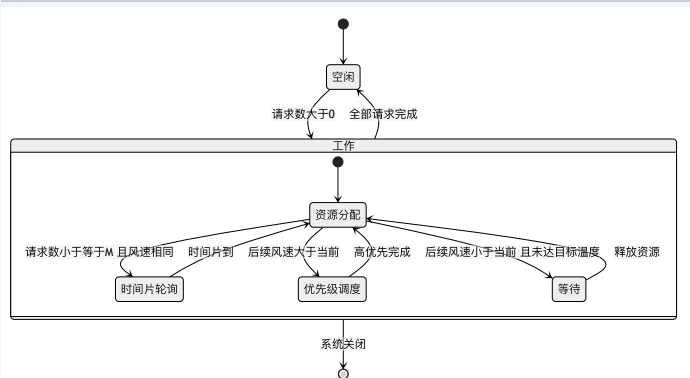
评分点：四对象齐全；带三参数的 generate；返回 confirmCode。

图 3 DBFacade 新增对象协作（持久化层标准范式）



对 应 类 ： DBFacade.insert/getMapper；XxxMapper implements IMapper；DBConnection.getConnection()。

图 4 分布式温控系统状态图（M=3，状态机大题）



评分点：一级空闲/工作；二级资源分配/调度/等待；调度策略三条条件；起始/结束状态。

图 5 自助挂号机状态图（2021 试题 Q249）

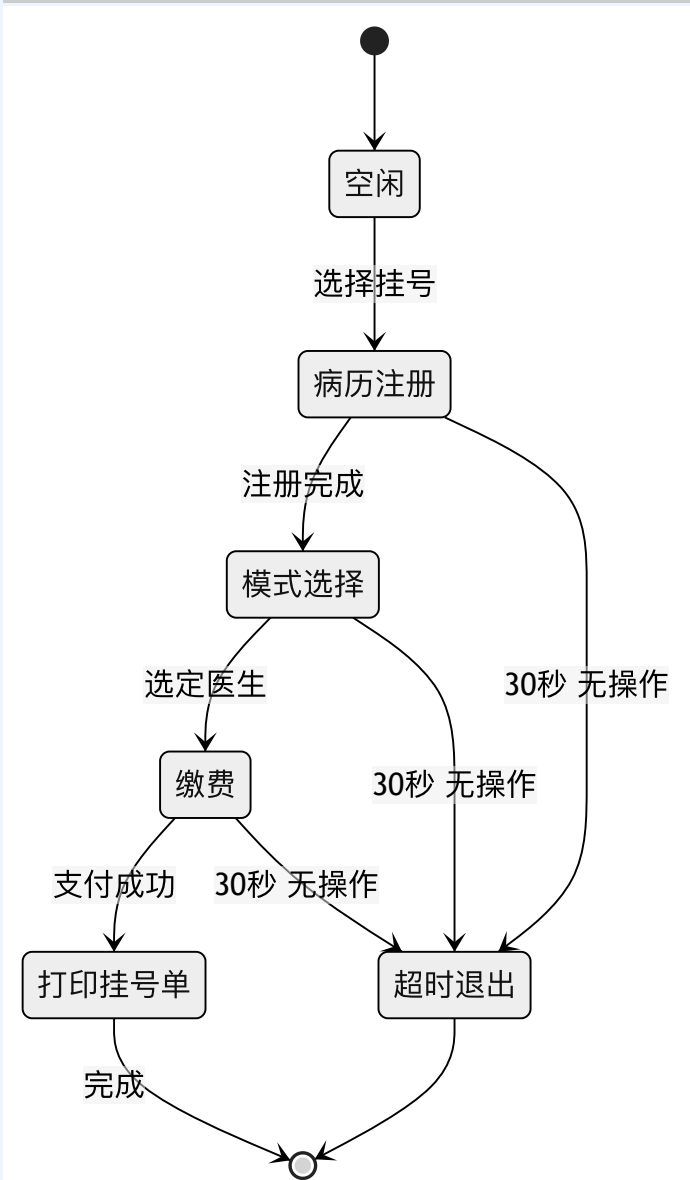


图 6 设计类图分层结构 (POS 机简化版)

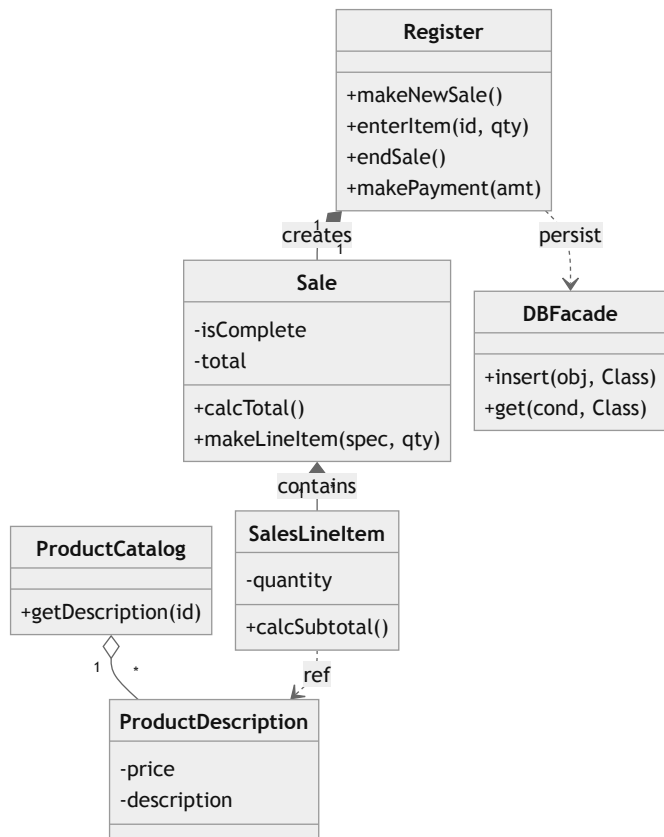
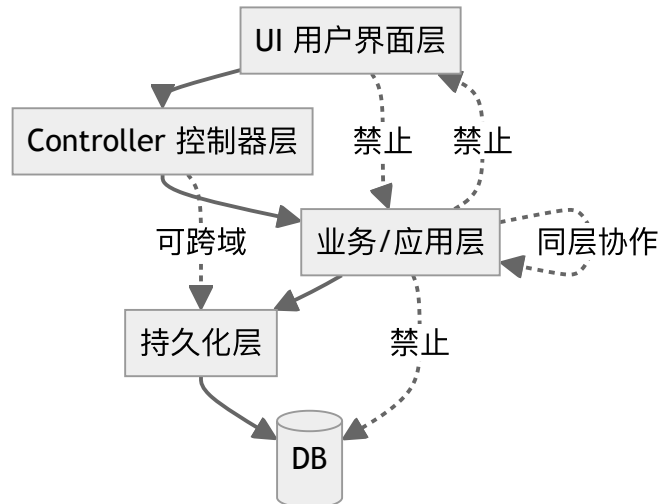


图 8 B/S 五层通信约束



七、历年真题汇总与详解（严格按教材修正）

7.1 判断题

判断 2007-2008 期末 #4

UML 中顺序图和协作图不仅能用来表示对象之间的动态行为，也能表示对象内部的状态变化。()

【标答】 ×

【解析】 顺序图与协作图属于交互图，描述对象间的消息交互；描述对象内部状态变化的是状态图 (State Diagram)。两者不可互相替代。

判断 2007-2008 期末 #9

在顺序图中，一个对象 A 发送了一条创建另一个对象 B 的消息，那么表明对象 B 具备了处理该条消息的职责。()

【标答】 ✓

【解析】 «create» 消息的接收者 B 需要提供构造方法处理该消息——这正是“接收消息即承担职责”原则的体现 (信息专家 + 创建者模式)。

判断 2007-2008 期末 #10

在进行软件类设计的过程中可以应用 GRASP 中的“信息专家”模式决定类中的方法。()

【标答】 ✓

【解析】 教材 7.3.4：根据消息的名称和参数为软件对象定义同名方法。

判断 2008-2009 期末 #8

面向对象设计中最关键的活动是找到对象并给对象分配职责。

【标答】 ✓

【解析】 教材 7.3.1 原文：OOD 最关键的活动之一是正确地给对象分配职责。

判断 2009-2010 期末 #10

面向对象设计中的开闭原则，要求对已有的软件类可以进行修改，而不能添加新的需求。

【标答】 ×

【解析】 OCP = 对扩展开放、对修改封闭。应通过添加新类/新方法扩展功能，不修改已有代码。题干表述恰好相反。

判断 2010-2011 期中 #4

面向对象设计中最关键的活动是找到对象并给对象分配职责。

【标答】 ✓

【解析】 同 2008-2009 #8。OOD 最关键的活动之一是正确分配职责。

判断 2010-2011 期中 #6

面向对象分析 (OOA) 和面向对象设计 (OOD) 分别采用不同的概念和表示法。

【标答】 ×

【解析】 OOA 与 OOD 采用一致的面向对象概念和 UML 表示法，无缝衔接。

判断 2010-2011 期中 #10

外观控制器不需要实现系统操作，但用例控制器需要。

【标答】 ×

【解析】 教材 7.3.2：不论是外观控制器还是用例控制器，都只接收系统事件消息，不实现系统事件请求的内容。

判断 2010–2011 期中 B #6

在顺序图中，对象 A 发送给了对象 B 一条消息，那么表明对象 A 具备了处理该条消息的职责。

【标签】×

【解析】接收消息的 B 才有处理该消息的职责（信息专家模式）。

判断 2010–2011 期中 B #10

设计模型中的交互图是决定系统静态结构的基础和前提。

【标签】√

【解析】教材 7.3：必须先得到动态结构（交互图），再归纳出静态结构（设计类图）。

判断 2011–2012 期末 B #2

OOA 和 OOD 分别采用不同的概念和表示法。

【标签】×

【解析】同 2010–2011 #6。OOA/OOD 一致表示法。

判断 2011–2012 期末 B #6

在协作图中，对象 A 接收了对象 B 一条消息，那么表明对象 B 具备了处理该条消息的职责。

【标签】×

【解析】接收者 A 才是职责承担者；题干把发送/接收方关系颠倒。

判断 2011–2012 期末 B #8

面向对象设计中最关键的活动是正确地给对象分配职责。

【标签】√

【解析】教材 7.3 原文。

判断 2011–2012 期末 B #10

用例控制器不需要实现系统操作，但外观控制器需要。

【标签】×

【解析】同 2010–2011 #10。两类控制器都不实现。

判断 2012–2013 期末 B #6

面向对象设计中，寻找对象职责的有效方法之一是绘制交互图。

【标签】√

【解析】交互图逐条画出对象间消息（= 方法调用 = 职责承担），是发现/分配职责的有效手段。

判断 2013–2014 期末 A #6

OO 设计中使用 UML 的交互图来表示数据的流动方向。

【标签】×

【解析】交互图描述对象间的消息交互，不是数据流动；表示数据流动是结构化分析的数据流图 DFD。

判断 2018–2019 期末 B #8

无论外观还是用例控制器，均无需实现系统事件请求的内容。

【标签】√

【解析】教材 7.3.2 原文结论。

判断 2023–2024 期末 A #4

UML 交互图中对象 A 给对象 B 发送了一条同步消息 M，则对象 A 具有功能 M。

【标签】×

【解析】接收消息的 B 才具有功能 M（7.3.4 信息专家 + 7.3.7 设计类图推导）。

判断 2023–2024 期末 A #5

UML 交互图中一个对象实例不可以给自己发送消息。

【标签】×

【解析】UML 允许 self-message（自身折返箭头），教材交互图实例中多次出现。

判断 2024–2025 期中 A #10

操作契的后置条件描述的是系统顺序图中的返回消息。

【标签】×

【解析】后置条件描述领域模型对象的状态变化（创建/消除、关联形成/消除、属性修改），不是 SSD 的《return》消息。

7.2 单选题

单选 2009–2010 期末 A #4

系统顺序图属于哪一个模型。

【标签】用例模型（教材 4.5.2 列为用例模型 4 要素之一：用例图、用例说明、SSD、操作契约）。注：设计模型中使用的是设计顺序图，与 SSD 不同。

【解析】教材 4.5.2 明确 SSD 属用例模型。原卷自带“设计模型”答案不正确——本题修正为用例模型。

单选 2009–2010 期末 A #9

设计模型所需要完成的工作中不包括（ ）。

【标签】操作契约。

【解析】操作契约属 OOA 阶段的使用例模型产物，是设计的输入而非设计模型本身。设计模型包括：确定类的方法与属性、绘制用例实现的顺序图、绘制设计级状态图、绘制设计类图。

单选 2010–2011 期中 A #10

面向对象设计模型中不包括（ ）。

【标签】A. 操作契约。

【解析】操作契约是 OOA 用例模型的附加内容；设计模型 = 动态结构（交互图）+ 静态结构（设计类图）+ 补充状态图。

单选 2010–2011 期中 B #9

GRASP 信息专家模式决定了对象的（ ）。

【标签】C. 方法。

【解析】教材 7.3.4：根据消息的名称和参数定义同名方法。属性来自领域模型，关系/名称不归信息专家直接决定。

单选 2010–2011 期末 B #10

下面关于面向对象设计描述错误的是（ ）。

【标签】D. 应先进行对象职责分配再做软件体系结构设计。

【解析】教材 7.1：先确定体系结构框架，再在各层上分配职责。其余 A/B/C 描述均符合教材。

单选 2011-2012 期末 B #4

UML 顺序图可以表示以下什么模型 ()。

【标答】D. 用例模型 (SSD) 与设计模型 (设计顺序图) 均可。

【解析】SSD 是用例模型 4 要素之一；设计顺序图用于设计模型描述用例实现。

单选 2011-2012 期末 B #9

面向对象设计原则中的开闭原则是指 ()。

【标答】C. 对扩展开放、对修改封闭 (只允许添加新功能/新类)。

【解析】教材 6.7.2 OCP 定义。

单选 2012-2013 期末 B #10

下面关于面向对象设计描述错误的是 ()。

【标答】D. 先静态后动态。

【解析】教材 7.3: 先动态 (交互图)、后静态 (设计类图)。D 把顺序说反。

单选 2013-2014 期末 A #10

下面关于面向对象设计描述正确的是 ()。

【标答】A. 业务逻辑层对象的设计依据应参考领域模型的概念类。

【解析】教材 7.2.2 原文。B 错 (OOA/OOD 一致表示法)；C 错 (创建者不限于动态创建)；D 错 (先动态后静态)。

7.3 填空题

填空 2023-2024 期末 A #3

用例模型由四个部分构成：用例图、用例说明、系统顺序图和 ()。

【标答】操作契约 (Operation Contract)。

填空 2023-2024 期末 A #5

面向对象设计中交互图的主要任务是确定 () 并为其分配功能。

【标答】软件对象 (或控制器对象)。

【解析】教材 7.3.5: 交互图核心任务是确定系统事件进入系统后被激活的软件对象，并为每个对象分配职责/方法。

7.4 简答题

简答 2008-2009 期末 A #3

阐述面向对象设计中依赖倒置原则的含义。

【标答】

依赖倒置原则 (DIP):

1. 高层模块不应依赖低层模块，两者都应依赖抽象；
2. 抽象不应依赖细节，细节应依赖抽象；
3. "倒置"双重含义——依赖关系倒置 + 接口所有权倒置 (客户类拥有抽象接口，服务类派生自该接口)；
4. 启发式规则：任何变量不持有指向具体类的指针/引用；任何类不从具体类派生；任何方法不覆盖基类已实现方法。

【解析】教材 6.7.4 DIP 是框架设计核心原则，提高高层模块可复用性。

简答 2009-2010 期末 A #3

面向对象的设计模型是什么？如何构建软件的静态和动态模型？

【标答】

设计模型：在 OOA 模型基础上加入实现约束 (架构、语言、数据库、复用)，可直接指导编码的 OOD 产物。包括：

- 静态结构模型：设计类图——属性、方法、可见性、关联、聚合、组合、继承、依赖、包组织。
- 动态行为模型：顺序图、协作图、状态图、活动图——对象交互、消息、状态变迁。

构建静态结构：①领域模型→设计类；②据顺序图补方法；③细化属性类型与可见性；④细化关系多重性与方向；⑤按 GRASP 分配职责，必要时引入设计模式；⑥按分层架构组织包。

构建动态行为：①每用例绘顺序图；②关键对象绘状态图；③复杂流程绘活动图；④与静态模型双向校验 (顺序图方法↔类的方法)。

简答 2010-2011 期中 B #2

简述用例模型、领域模型及设计模型之间各种图的使用关系。

【标答】

- 领域模型：UML 类图表达概念类及其关系 (必要时活动图辅助)。
- 用例模型：4 要素——用例图、用例描述、系统顺序图、操作契约。
- 设计模型：动态 (交互图：顺序图/协作图) + 静态 (设计类图、组件图、包图)。

关系链：领域模型 → (概念类→软件类) → 用例模型 SSD → (系统事件→控制器接收→对象协作) → 设计动态结构 → (消息→方法/属性) → 设计静态结构。必要时反哺领域模型修订。

简答 2010-2011 期末 B #4

(2) 领域模型的类图与设计模型的类图有什么不同？

【标答】

维度	领域模型类图	设计模型类图
类性质	概念类	软件类 (直接映射源代码)
包含内容	类名、关键属性、关联；不含方法	属性 (含类型)、方法 (含参数与返回值)、关系、可见性
来源	名词短语分析法	领域模型转换 + 用例实现新增 (控制器、DBFacade、UI 等)
抽象层次	高层、与实现无关	接近实现

简答 2011-2012 期末 B #1

OOA 阶段与 OOD 阶段有什么不同？相对于结构化需求分析与设计，OO 方法有什么优势？

【标答】

不同：分析针对问题域，把需求转化为分析模型；设计须考虑实现问题，产生可编码的设计模型。

优势：OOA 与 OOD 采用一致的面向对象概念与 UML 表示法，OOA 到 OOD 不存在转换，紧密衔接；结构化方法中分析与设计之间存在鸿沟。

简答 2012-2013 期末 B #1

面向对象设计需要做哪些工作？这些工作对应的成果模型是什么？

【标答】

三方面工作：

1. 软件体系结构设计 → 包图；
2. 用例实现方案设计：
 - 动态结构 → 顺序图/协作图；
 - 复杂对象 → 状态图；
 - 静态结构 → 设计类图；
3. 用户界面设计（可与上并行）；
4. 详细设计：方法内部逻辑 → 活动图/程序流程图。

简答 2013-2014 期末 A #5

请列举出面向对象设计过程中常用的五层软件分层体系结构，并说明模型层次化的好处。

【标答】

五层：用户界面层、控制器层、业务/领域层、持久化层、系统层。

好处：①增加软件健壮性，易于扩展和维护；②可移植性；③减少类间耦合。

简答 2018-2019 期末 B #2

给出面向对象设计所必须的已知条件？以及设计结果是什么？

【标答】

已知（输入）：用例模型（4 要素：用例图、用例描述、SSD、操作契约）、领域模型、软件体系结构框架、面向对象设计原则与 GRASP 等设计模式。

设计结果（输出）：动态结构（交互图、状态图）+ 静态结构（设计类图、组件图、包图）。

简答 2024-2025 期中 A #3

请说明用例模型中系统顺序图中的指令与操作契约的关系。

【标答】

1. 一一对应：SSD 中每条由角色发给系统的同步消息（系统事件）对应一个操作契约。
2. 派生关系：操作契约在 SSD 基础上编写——先识别系统事件，再对复杂事件编写契约。
3. 关注点互补：SSD 关注“消息的形状（操作名 + 参数）+ 返回结果”；操作契约关注“执行后领域模型对象的状态变化”（创建/消除、关联形成/消除、属性修改）。
4. 同属用例模型 4 要素。

7.5 应用题

应用 2010-2011 期中 A #3 (10分)

根据下面顺序图确定类 B 中的方法，并根据对象 b 与对象 c,d,e 的消息，给出类 B 中 M_AB 方法的 Java 或 C++ 代码。

【标答】

类 B 至少包含方法 M_AB()（接收消息 M_AB 的职责），其内部按顺序调用 c.M_BC()、d.M_BD()。

```
class B {
public:
    void M_AB() {           // 本身方法定义
        // ...
        c.M_BC();           // 调用对象 c 的方法
        // ...
        d.M_BD();           // 调用对象 d 的方法
        // ...
    }
};
```

【解析】教材 7.3.7：发送给类 X 的所有消息集合就是类 X 必须定义的大多数操作；对象内部发出的消息即方法体内的调用。语法不严，意思对即可。

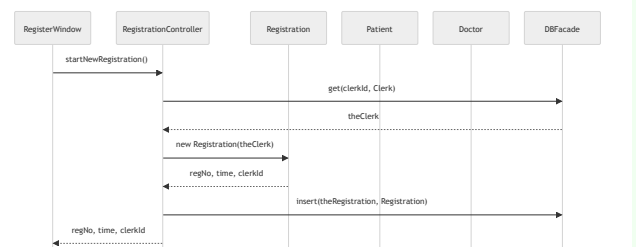
7.6 综合题（含 Mermaid 重绘）

综合 2007-2008 期末 A · 问题三 (15分)

"StartNewRegistration"系统消息：根据 GRASP 信息专家模式绘制设计模型的交互图并给出对象所对应类的方法。

【标答】

参与对象：RegisterWindow(UI)、RegistrationController、Registration、Patient、Doctor、DBFacade、DB。



对应类的方法：

- RegistrationController.startNewRegistration()——控制器模式
- Registration(constructor)——创建者模式（由 Controller 创建）
- Registration.getRegNo()/getTime()/getClerkId()——信息专家
- DBFacade.get(cond, Class) / insert(obj, Class)

综合 2008-2009 期末 A · 问题三 (15分)

"nextPatient(doctorID)"系统消息：根据 GRASP 信息专家模式绘制设计模型交互图并给出方法。

【标答】

对象与消息见 §六 图 1。

方法：NextPatientController.nextPatient(doctorID)、PatientQueue.getNext()、Patient.getBasicInfo()、DBFacade.get(cond, Class)。

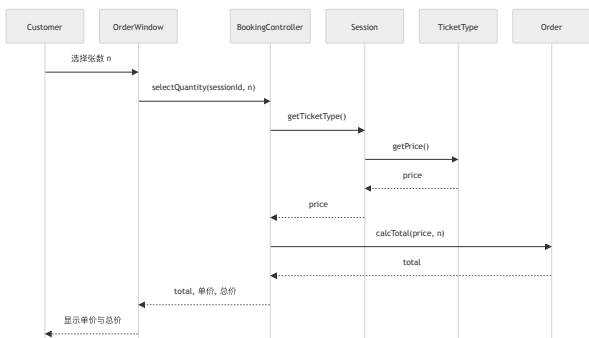
GRASP 依据：NextPatientController 是 用 例 控 制 器；PatientQueue 知道队列顺序（信息专家）；Patient 知道自身基本信息（信息专家）；PatientQueueDAO 负责按 doctorID 查询队列。

综合 2009-2010 期末 A · 问题三 (9分)

网上订票订座系统：根据场景第 5、6 条（系统显示单价，顾客选择张数；系统给出总价）及领域模型，给出该系统消息的设计顺序图。

【标答】

系统事件：selectQuantity(sessionId, ticketCount)；系统计算总价返回。



GRASP：BookingController 用例控制器；TicketType 是单
价的信息专家；Order 拥有计算总价所需的张数与单价，负
责 calcTotal()。

综合 2010-2011 期中 A · 问题三 (10分)

电子商务网站场景第 8、9 条（输入网银账号、身份证号、密
码；系统据账号+身份证+订单号生成确认码），给出该系统消
息设计模型的交互图。

【标答】

见 §六 图 2。

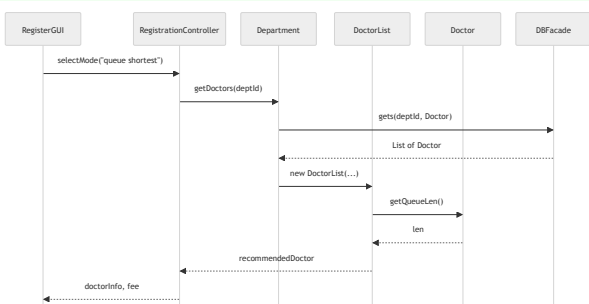
评分重点：四对象齐全（Customer、ShoppingController、
Order、ConfirmationCodeGenerator）；获取订单号及返
回；generate 消息带三参数；返回 confirmCode。

综合 2010-2011 期中 B · 问题 3 (10分)

根据 GRASP 的控制器模式决定该《医院自助挂号管理系统》
的控制器对象，并给出“根据医生的排队情况确定医生”系统事
件的设计模型顺序图。

【标答】

控制器选择：系统事件较多、用例独立，选用例控制器
RegistrationController。

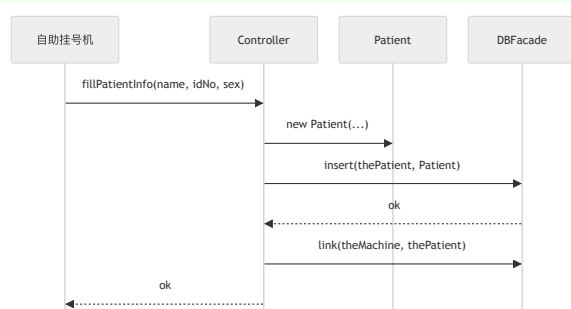


综合 2011-2012 期末 B · 问题一 (20分)

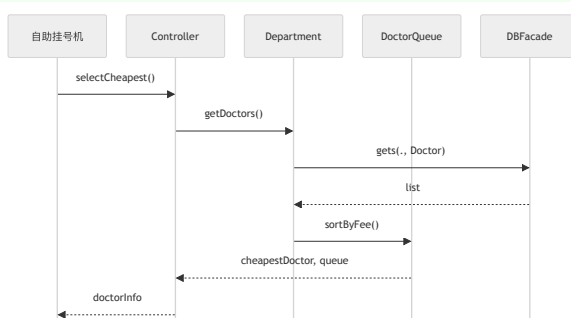
某医院自助挂号系统：针对系统事件_1（填写病历注册信息）
和系统事件_2（选择挂号费最少），分别画出软件设计模型中
的顺序图（8分）；按信息专家模式确定每个对象的方法定义
（4分）。再以自助挂号机为对象，描述其状态迁移图（8
分）。

【标答】

事件_1 顺序图：



事件_2 顺序图：

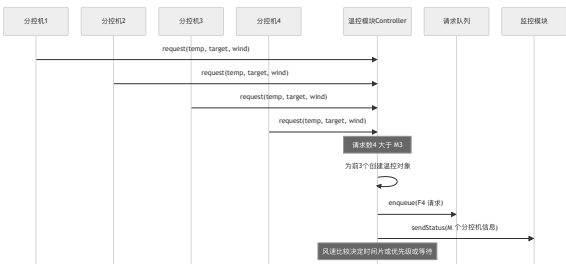


方 法 ： Controller.fillPatientInfo/selectCheapest；
Patient.setters)；Department.getDoctors；
DoctorQueue.sortByFee；DBFacade.insert/get/gets。
状态图见 §六 图 5。

综合 2012-2013 期末 B · 问题一 (20分)

分布式温控系统 M=3：以分控机温度请求作为系统事件，画出系统事件对应的交互图（提示：以温控模块作为控制器对象，至少考虑 4 个分控机对象的温度请求）。

【标答】



GRASP：温控模块是用例控制器；分控机是信息专家（知当前温度）；请求队列是信息专家（知排队顺序）；监控模块是协调对象。
状态图见 §六 图 4。

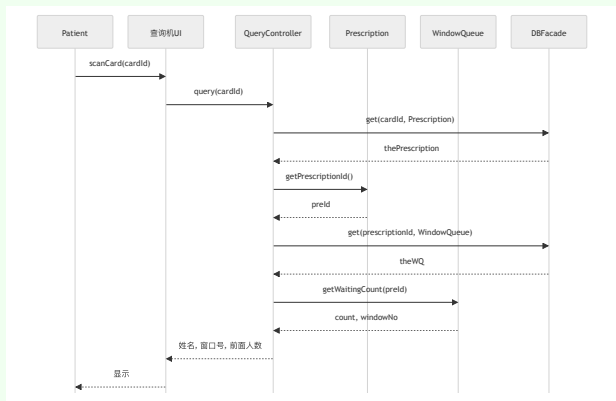
综合 2013-2014 期末 A · 综合 4 (10分)

医院取药告知系统：针对病人使用查询机查询，确定操作契约以及该查询事件对应的设计模型交互图（按分层结构明确软件对象名称与消息）。

【标答】

操作契约：

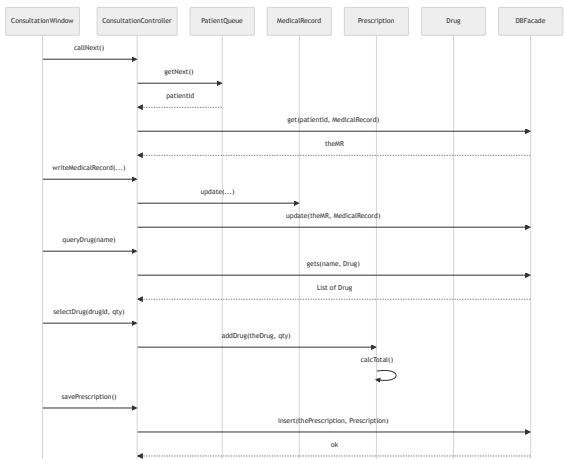
- 前置条件：病人已缴费，持就医卡。
- 后置条件：①查询机与处方实例建立关联（关联形成）；②查询机与窗口队列实例建立关联；③窗口队列属性被读取。



综合 2015-2016 期中 A · 问题 7

医院就诊管理系统（B/S 分层 GUI/控制器/应用逻辑/持久化/DB）：给出"问诊"用例中"呼叫病人、查询病历、书写病历、书写处方（药品查询、选择药品、保存）"系统事件的实现过程顺序图。

【标答】

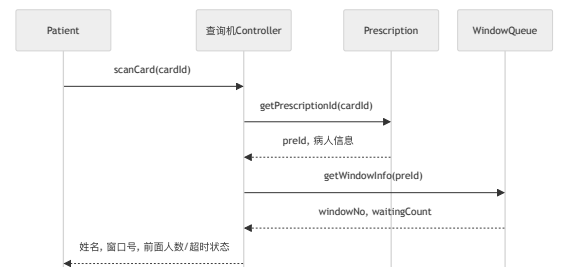


分层职责：UI 仅输入输出；Controller 协调；MedicalRecord 维护病历；Prescription 维护明细并算总价（信息专家）；Drug 保存药品信息；DBFacade 封装持久化。

综合 2018-2019 期末 B · 综合 3 (15分)

医院取药告知系统：以查询机为控制器对象设计病人使用查询机获得排队信息的顺序图（假定该查询不需要数据库查询）。

【标答】



说明：本题不需要 DB 查询，数据由 Prescription、WindowQueue 内存对象提供（信息专家）。

亡者农药自助系统"刷脸充值"用例：(5) 系统顺序图；(6) 每条指令的操作契约；(7) 据步骤 2、3 及操作契约给出软件设计模型交互图。

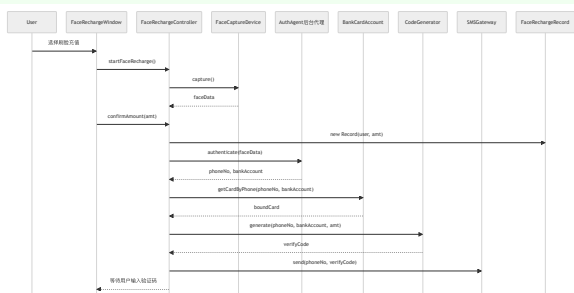
【标答】

(5) SSD：Actor → 系统：startFaceRecharge → confirmAmount(amt) → 输入验证码 → 系统完成。

(6) 操作契约核心后置条件：

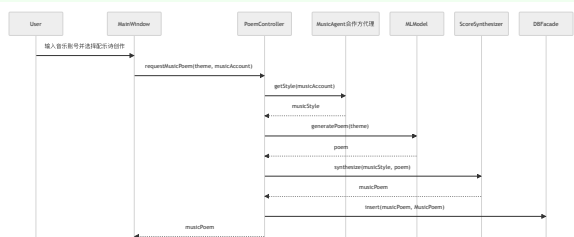
- startFaceRecharge：FaceRechargeRecord 实例创建；与 User 建立关联。
- captureFace：FaceData 属性被修改（采集人脸数据）。
- getBoundAccount（系统内部）：返回手机号、网银账号。
- generateVerifyCode：VerifyCode 实例创建；属性修改；与 Record 关联。
- confirmCode：Record 状态属性被修改；User 积分属性被修改。

(7) 设计交互图（步骤 2、3）：



GRASP：FaceRechargeController 用例控制器；FaceCaptureDevice 是采集专家；AuthAgent 是外部认证代理（信息专家——知绑定关系）；CodeGenerator 是验证码专家；FaceRechargeRecord 由 Controller 创建（创建者）。

写诗机器人"小红"·场景三：用户输入音乐账号后，系统先获取听歌风格，再用主题句生成独创诗，最后合成配乐诗返回。给出软件设计模型的交互图，说明软件对象功能（控制器模式 + 信息专家模式）。

【标答】

对象功能：PoemController 用例控制器；MusicAgent 外部代理信息专家（知风格）；MLModel 信息专家（知如何生成）；ScoreSynthesizer 协调合成；DBFacade 持久化。

校医院自助挂号机：给出第 2（选择挂号）和第 3（选择科室）条指令的操作契约，并给出对应交互图。

【标答】

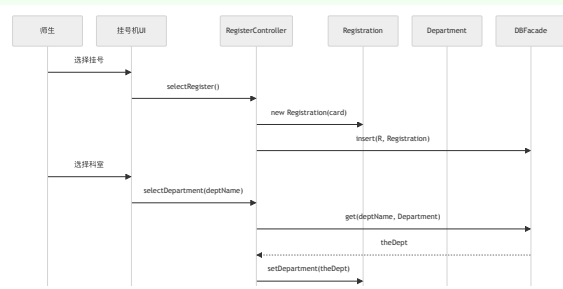
第 2 条 selectRegister() 契约：

- 前置：一卡通已识别。
- 后置：创建 Registration 实例；与 Card/User 建立关联；初始化时间、状态属性。

第 3 条 selectDepartment(deptName) 契约：

- 前置：Registration 已创建。
- 后置：Registration 与 Department 建立关联；Department 属性（科室名）被修改。

交互图：

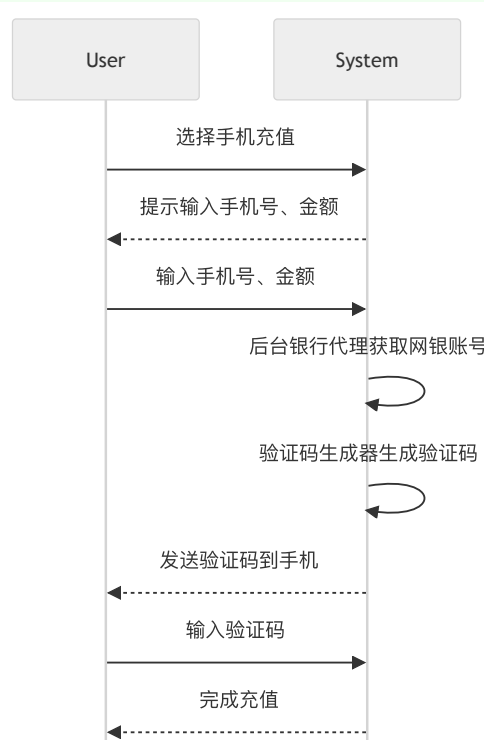


架构为 GUI、控制器层、应用逻辑层，给出第 3 条指令（选择科室）的交互图：确定控制对象及应用逻辑层的软件对象。

【标答】

同上题"选择科室"流程，参与对象：**RegisterController**（控制器层）、**Registration**、**Department**（应用逻辑层）；GUI 层为挂号机界面；按题意不要求持久化层。Controller 接收 selectDepartment，向应用逻辑层 Department 查询并修改 Registration 关联。

学生食堂一卡通"手机充值"场景 B：绘出系统顺序图。

【标答】

手机充值场景 B 第二条指令的操作契约。

【标答】

指令: inputPhoneAndAmount(phoneNo, amount)

- 前置: 用户已登录、手机已绑定银行卡。
- 后置: ①RechargeRequest 实例创建 (实例创建);
②RechargeRequest 与 User 建立关联 (关联形成);
③RechargeRequest 的 phoneNo、amount 属性被修改 (属性修改)。

八、A4 双栏 Cheat Sheet (考前 5 分钟速查)

OOD 总框架

- 输入: 用例模型 + 领域模型 + 架构 + 原则/模式
- 输出: 动态 (交互图/状态图) + 静态 (设计类图/包图/组件图)
- 口诀: 先体系结构、再动态、后静态
- OOD 最关键: 正确给对象分配职责

五层架构 (必背顺序)

- UI → 控制器 → 业务 → 持久化 → 系统/DB
- 同层可协作, 相邻层单向
- 禁: 业务调 UI; GUI 直访 DB
- 好处: 健壮、扩展、维护、移植

GRASP 三大模式

- 控制器**: 谁接收系统事件 → 控制器层
- 创建者**: B 聚合/记录/密切用/有初始化数据 → B 创建 A
- 信息专家**: 拥有信息的类 → 拥有方法
- 铁律: 接收消息者 = 有该方法

外观 vs 用例控制器

- 外观: 系统/设备名, 事件少时
- 用例: <用例>Controller, 臃肿时
- 共同点: 只接收/委派, **不实现**
- 同用例同控制器实例

SSD vs 设计顺序图

- SSD: 用例模型, Actor ↔ 系统黑盒
- 设计顺序图: 设计模型, 多生命线
- SSD 4 要素: 用例图/说明/SSD/契约
- 操作契约 ≠ 返回消息, 是状态变化

持久化层五件套

- DBFacade: 统一入口, 5 操作
- IMapper: CRUD 接口
- XxxMapper: 实现 IMapper
- DBConnection: 连接管理
- DB: 实际存储

DBFacade 5 方法

- boolean insert(obj, Class)
- boolean delete(cond, Class)
- boolean update(obj, Class)
- Object get(cond, Class)
- List gets(cond, Class)
- + private getMapper(Class)

设计类图 6 步

- ①扫描识别类
- ②加属性
- ③据交互图加方法
- ④补类型
- ⑤加关联/导航 (A→B: 发消息/创建/连)
- ⑥细节 (可见性/方法体)

方法推导 3 铁律

- 接收消息的类有该方法
- create/get/set 一般忽略
- 多对象消息 → 容器对象的方法
- 组合/聚合 → 属性或集合

综合题 6 步法

- ①找系统事件
- ②选控制器 (外观/<用例>Controller)
- ③找业务对象
- ④分配职责 (GRASP)
- ⑤画图: GUI→C→业务→DBFacade
- ⑥说明 GRASP 依据

顺序图生命线左→右

- Actor / GUI
- Controller
- 业务对象 1..n
- DBFacade / DAO / 外部代理
- DB / 外部系统

状态图要素

- 起始●/结束◎
- 一级状态 (空闲/工作)
- 二级子状态 (调度/等待/分配)
- 迁移条件写箭头
- 覆盖超时/异常/调度

易错判断题

- OOD 关键 = 分配职责 (√)
- OOA/OOD 表示法一致 (√ 说法; 题说不同=×)
- 控制器都“不实现” (√)
- 接收者才有职责 (A 发 B, B 有)
- OCP = 扩展开放/修改封闭
- 交互图 ≠ 数据流图
- 对象可发自消息 (√)

OOA vs OOD

- OOA: 问题域、概念类、无方法
- OOD: 实现、软件类、含方法
- 两者一致表示法、无缝衔接
- 设计模型 ≠ 操作契约

大题必拿分点

- 对象齐全 (≥4)
- 消息带关键参数
- 返回值虚线
- 体现创建和持久化
- 图后写对象职责
- 分清 SSD vs 设计图

POS 机要点 4 系统事件： makeNewSale/enterItem/endSale/makePayment - 外观控制器：Register - Sale 创建 SalesLineItem（创建者） - Sale.calcTotal()（信息专家） - ProductCatalog 包含 ProductDescription
考试系统要点 20 用例→用例控制器 - LogonController / TakeTestController - TestPaper.getScore()（信息专家） - Question.getSubScore()（抽象方法） - Rule 创建 RuleItem（创建者）

教材关键原文 "为得到静态结构，必须先得到动态结构" "接收消息的对象一定具有处理该消息的职责" "OOA 与 OOD 采用一致的表示法" "控制器只接收系统事件消息，没有实现系统事件请求的内容"
startUp 原则 - 最后设计 startUp - 选容器/根/外观控制器 - POS：Store 为根 - 应用发 create → 创建初始领域对象